Coding Project 1: Take The ‘A*’ Train

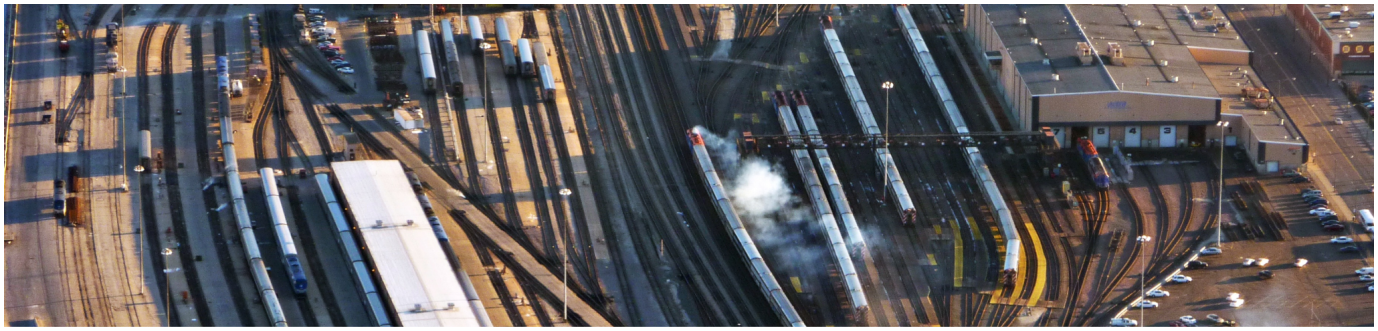
Spring 26

Goals: Uninformed Search, Informed Search, Heuristic Design
This assignment is adapted from Dr. Keith Decker and Dr. Jermey Keffer at the University of Delaware.

If clarifications are needed, they will be documented on this page.

Last Updated 2/2/26
Changelog:

- For possible actions, Right and Left, clarified that the engine itself can move if it is the rightmost/leftmost car on the track.



Introduction

Consider the problem of maneuvering railroad cars in a train yard. Many cars must be assembled into a train of cars in a given order, but the cars start out dispersed throughout the yard, and can only move in prescribed ways on existing tracks.

Below is an example of a small train yard with sufficient connectivity to arrange railroad cars into any desired sequence. The different sections of the track have been numbered from 1 to 6. The figure also shows the starting position for a small train, where each car is represented by a lowercase letter, and the engine is represented by an asterisk “*”. Finally, we show the same cars in the goal position, with the cars lined up in order for a departure to the west.

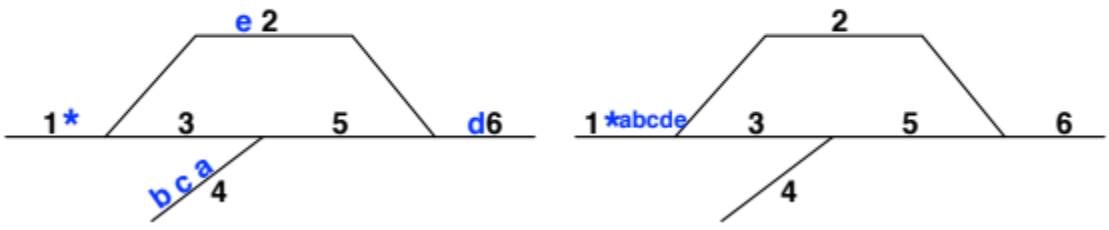


Figure 1: The initial and goal states of *Yard 5*.

You will be writing a small program that when given a yard, an initial configuration of cars, and a goal configuration, will use a search algorithm to produce a set of actions that transform the initial state into the

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

goal state. While this is a toy problem, this is a basis for many real world logistics and optimization problems (e.g. stacking shipping containers).

Logistics

Starter Code

Your HW is structured as follows:

```
coding_0/
├─ COLLABORATION.MD
├─ tests/
│  ├─ [tests...]
├─ yards/
│  ├─ yard_X_DIFFICULTY.json
│  ├─ etc…
├─ test.py
├─ YardSearcher.py
├─ README.md
```

`COLLABORATION.MD`:

- An empty file where you will put your collaboration statement, listing anyone or anywhere you got help from. If you truly did not get help from anywhere or anyone, you must write that in your collaboration statement. **You will lose points if this is not completed.**

`tests/`:

- A set of tests for this assignment. The possible tests are [q1, q2, q3, q4, q6]

`yards/`:

- A folder containing different problem statements of a yard in `.json` format.
- Each file contains a `Yard yard`, an initial `State init` and a goal `State goal`. See **Setup 1: Problem Representation** for more info on this representation.
- The yards have various difficulties
  - An **Easy** yard is a trivial case which you can verify very quickly, perfect for testing.
  - A **Medium** yard is a proper puzzle that you as the human might take some time to complete, but a proper implementation should solve relatively instantly.
  - A **Hard** yard is reaching on the edge of what you could quickly do as a human, and you may start to notice an uniformed search algorithm taking some time here (e.g. a few seconds). An informed search algorithm should be very quick however.
  - An **Extreme** yard is only feasibly possible with an informed search algorithm and heuristics and even *then* will take some time to complete (e.g. around a minute). You will be waiting for a *long long* time if you use an uninformed search algorithm (that is if you don't run out of memory).

`test.py`

- This file contains test cases for the functions you are going to implement in this assignment, much like prior assignments. **These will NOT be the only tests you are tested on**, and it is encouraged that you add your own test cases.

`YardSearcher.py`:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- In this file is some starter code for reading in descriptions of the yard problems, as well as function stubs that you will fill in to solve these problems.

README.md
- Some basic information on running `YardSearcher.py`.

Grade Breakdown

Feature	Points	Graded By
Problem 1: Possible Actions	5 - 5 given tests, each worth 1 point.	Autograder
Problem 2: Result	10 - 5 given tests, each worth 2 points.	Autograder
Problem 3: Apply Action Sequence	5 - 5 given tests, each worth 1 point.	Autograder
Problem 4: Uninformed Search	20 - 3 given tests, each worth 4 points - 2 hidden tests, each worth 4 points.	Autograder
Problem 5: Heuristic Design	20	Manual
Problem 6: A* Search	30 - 3 given tests, each worth 5 points - 3 hidden tests, each worth 5 points	Autograder
Style and Readability	5	Manual
Collaboration Statement	5	Manual
Total	100	N/A

Submission Instructions

In gradescope please upload **ONLY** the following files:
- `YardSearcher.py`
- `COLLABORATION.MD`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Background

Problem Representation

For the purposes of this assignment, we'll be using the following problem representation.

`Yard` `(List[List[int]]):`
- A connectivity list of all the switches in the yard.
- The right edge of track 1 connects to the left edge of track 2 and 3, both [1, 2] and [1, 3] are in our lists.
- Contrary, while 3 and 4 are adjacent, one can not move a car directly from 3 to 4, so [3,4] is not in our connectivity list.

`State` `(List[List[str]]):`
- A list of size *n*, where *n* is the number of tracks in our yard. Each car is represented by a lowercase letter.
- The engine is represented by '*'.
- When multiple cars are on a track, they are listed from left to right.
- An empty list means there are no cars on that track.

Example:

Recall we can diagram our yard, our initial, and goal states as follows:

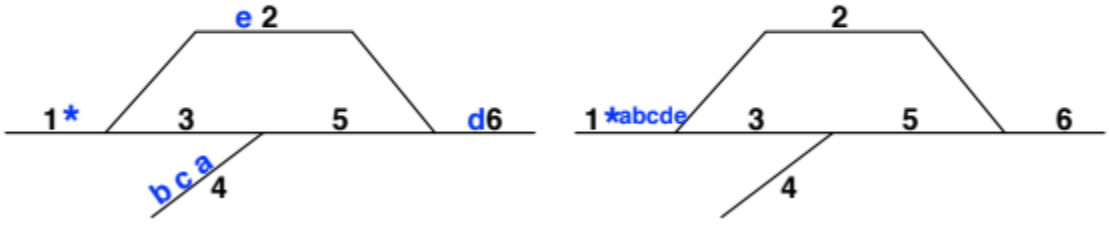


Figure 1 (again): The initial and goal states of *Yard 5*.

This is represented in `yard_5_HARD.json` as:

```
{
      "yard": [[1, 2], [1, 3], [3, 5], [4, 5], [2, 6], [5, 6]],
            "init": [["*"], ["e"], [], ["b", "c", "a"], [], ["d"]],
            "goal": [["*", "a", "b", "c", "d", "e"], [], [], [], [], []]
}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

A Note On Representation:

Internally, you may represent these states in any way you wish - in fact we encourage it as a good representation may make manipulating the state space easier. Feel free to build auxiliary classes and functions. However, for the purposes of testing, the output of the functions we ask you to modify MUST be in this format. **All your code must be in `YardSearcher.py`.**

Possible Actions

For this assignment, we'll be using the following representation of actions possible in the yard problem. There are only two actions, moving a single car **RIGHT** or **LEFT**. We give a description below, but see `q1_test_possible_actions.py` if you want to see an example in code.

`Action` `(tuple[str, int, int])`

- An action is a 3-tuple representing a movement of a car between two tracks
- The string can only take on the values **"RIGHT"** or **"LEFT"**
- The two integers must take on values corresponding to tracks present in a **`Yard`**.

Rightwards Move:

`("RIGHT", x, y)`



Figure 3: `("RIGHT", 1, 2)` : Move rightmost car FROM track 1 RIGHTward TO become leftmost car on track 2. "Move one car RIGHT from track 1 to track 2"

- Move one car rightward from the right end of track x to the left end of track y.
- If the connectivity list contains the sublist [x, y] and either track x or track y contains the engine, then the rightmost car on track x (including the engine itself) can be removed from track x and placed at the leftmost position of track y.
- Only one symbol may be moved at a time.
- Examples of applying a **RIGHT** move on different **`States`**:
  - `INIT: [ ['*'], ['e']]`
  - `APPLY: ("RIGHT", 1, 2)`
  - `RESULT: [ [], ['*', 'e']]`

  - `INIT: [ ['a','*', 'b'], ['c','d']]`
  - `APPLY: ("RIGHT", 1, 2)`
  - `RESULT: [['a','*'], ['b','c','d']]`

  - `INIT: [ ['a'], ['*', 'e']]`
  - `APPLY: ("RIGHT", 1, 2)`
  - `RESULT: [ [], ['a', '*', 'e']]`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Leftwards Move:

`("LEFT", y, x)`



Figure 3: `("LEFT", 2, 1)` : Move leftmost car FROM track 2 LEFTward TO become the rightmost car on track 1 - "Move one car LEFT from track 2 to track 1".

- Move one car leftwards from the left end of track y, to the right end of track x.
- If the connectivity list contains the sublist [x, y] and either track x or track y contains the engine, then the leftmost car on track y (including the engine itself) can be removed from track y and placed at the rightmost position of track x.
- Only one symbol may be moved at a time.
- Examples of applying a **LEFT** move on different **`States`**:
    - `INIT: [ ['*'], ['e']]`
    - `APPLY: (LEFT, 2, 1)`
    - `RESULT: [ ['*','e'], []]`

    - `INIT: [ ['a','*', 'b'], ['c','d']]`
    - `APPLY: (LEFT, 2, 1)`
    - `RESULT: [ ['a','*', 'b', 'c'], ['d']]`

    - `INIT: [ ['a'], ['*', 'e']]`
    - `APPLY: (LEFT, 2, 1)`
    - `RESULT: [ ['a','*'], ['e']]`

Notes:

No other actions are allowed. Cars cannot move without using the engine, jump over other cars, or teleport from one track to another. Only one car can be pushed at a time. In this version of the problem, when the engine pushes a car, the engine then returns to its original position.

Programming

Complete the following problems. Each problem builds off the previous. Pseudocode for some of these functions can be found in Chapter 3 of the textbook.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Problem 1: Possible Actions [5 Points]

In `YardSearcher`, complete the function `possible_actions`, which consumes a state and - using `Yard self.yard -` produces a `List[Actions]` that represent all possible actions that can be taken in that state.

Test your function with the `q1` tests.

Problem 2: Result [10 Points]

In `YardSearcher`, complete the function `result`. This function consumes an `Action` and a `State`, and produces a different `State` : which is the state of the world after carrying out the specific `Action`.

Be careful that you don't modify the input state! Be sure to understand the difference between a shallow and deep copy in python.

Test your function with the `q2` tests.

Problem 3: Apply Action Sequence [5 Points]

In `YardSearcher`, complete the function `apply_action_sequence`. This consumes a `List[Actions]`, an initial `State`, and returns the `State` that occurs after applying the list of actions in sequence.

Test your function with the `q3` tests.

Problem 4: Uninformed Search [20 Points]

In `YardSearcher`, implement an uninformed search algorithm in `uninformed_search`.

Your search algorithm function should take in an initial `State` and a goal `State`, and return a `List[Action]` that, when these actions are applied to the initial state, results in the goal state. It should also return an `int` representing the number of nodes that you explored/expanded during the search algorithm (i.e. the nodes that you have taken off of the queue and generated successors for. It is **NOT** the number of nodes left on the frontier.)

Your search should be **optimal** and find one of the shortest possible sequences of actions. There can be multiple shortest paths for a given problem to a solution, but you only need to find one. **Describe the search algorithm you are using in a comment.**

Test your function with the `q4` tests. All of the problems should be solved relatively instantly. **If you notice your algorithm exploring a lot more nodes than in the test cases, are you making sure you're keeping track of what states you've seen before?**

Now, on your own, try using your uninformed search algorithm on `yard_5_HARD.json`. You should notice a bit of a slowdown as your computer tries to solve it, but it should be completed in a few seconds.

You are most likely not going to get a solution with uninformed search on `yard_6_EXTREME.json` (and will probably run out of memory before you get anywhere close). But you can certainly *try*.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Problem 5: Heuristic Design [20 Points]

Design an admissible heuristic for the train yard problem. **Give a short, informal argument (3 to 5 sentences) on why your heuristic is admissible.** It does not have to be a rigorous mathematical proof, but the reasoning must be sound and your language has to be clear and concise - like you're trying to explain it to a coworker. Then, implement your heuristic function in `heuristic_function`.

We won't be grading you on how good your heuristic is, just that it is admissible and non-trival (e.g. your heuristic cannot be 0 or a constant). **Focus on correctness first!** Then, if you're looking for a challenge, see if you can come up with a better heuristic.

Problem 6: A* Search [30 Points]

Using your heuristic you created in Problem 5, implement A* search for this problem in `astar_search`. Much like the uninformed search, it should return a list of Actions as well as the number of nodes you explored/expanded during the search algorithm.

You should use a **priority queue** to manage your frontier. I would recommend using Python's built-in heapq library.

As people may have different heuristics, the number of nodes explored in your informed search will be different. But you should see ***roughly* an order of magnitude decrease** in the number of nodes explored in order to find an optimal solution. For example, in **Yard 4** with a simple heuristic I went down from exploring 10k nodes to around 3k nodes.

Run and check the `q5` tests for the upper bound of how many nodes these algorithms should be expected to explore. If your numbers are far larger, you may have a problem. **Again, focus on correctness and optimality first.**

You'll notice that Yard 5 is solved relatively quickly now.

You are now ready to attempt Yard 6. With a relatively simple heuristic I was able to get an optimal path in less than a minute (though I have a pretty strong computer so yours might take a little longer). **There is a 5 minute timeout on the Gradescope for Problem 6. If you find your program takes longer than 5 minutes on any of the yards: there is most likely some inefficiency in the core of your search algorithm which you'll need to address.**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF